worked around this limitation for filters by using `[TypeFilter]` or `[ServiceFilter]`, but there's no such solution for `ValidationAttribute`.

Instead, for validation attributes you must use the service locator pattern. As I discussed in chapter 9, this antipattern is best avoided where possible, but unfortunately it's necessary in this case. Instead of declaring an explicit dependency via a constructor, you must ask the DI container directly for an instance of the required service.

Listing 32.9 shows how you could rewrite listing 32.8 to load the allowed currencies from an instance of `ICurrencyProvider` instead of hardcoding the allowed values in the attribute's constructor. The attribute calls the `GetService<T>()` method on `ValidationContext` to resolve an instance of `ICurrencyProvider` from the DI container. Note that `ICurrencyProvider` is a hypothetical service that would need to be registered in your application's `ConfigureServices()` method in Startup.cs.

Listing 32.9 Using the service-locator pattern to access services

```
public class CurrencyCodeAttribute : ValidationAttribute
{
    protected override ValidationResult IsValid(
        object value, ValidationContext context)
    {
        var provider = context                                          ❶
            .GetRequiredService<ICurrencyProvider>();                   ❶
        var allowedCodes = provider.GetCurrencies();                    ❷

        if(value is not string code                                     ❸
            || !_allowedCodes.Contains(code))                           ❸
        {                                                               ❸
            return new ValidationResult("Not a valid currency code");   ❸
        }                                                               ❸
```